Indian Institute of Technology Indore

CS 216

Bitcoin Transaction Lab

Assignment Report

Legacy vs SegWit Transactions

Team Members

Name	Roll Number
Harsh Bhalla	240003033
Hardik Sanjeev Hazari	240003032
Abhinav Jain	240003003
Jatin Singh	240003035

Contents

1 Environment Setup **1**

2 Part 1: Legacy P2PKH Transactions **1**
- 2.1 Code . . . 1
- 2.2 Execution Walkthrough . . . 1
- 2.3 Script Analysis (P2PKH) . . . 2
  - 2.3.1 Structure of `scriptPubKey` (Locking Script) . . . 2
  - 2.3.2 Structure of `scriptSig` (Unlocking Script) . . . 2
  - 2.3.3 Script Evaluation: Unlocking + Locking Concatenation . . . 2
  - 2.3.4 btcdeb Validation Steps . . . 3
  - 2.3.5 Signature Validation . . . 3

3 Part 2: SegWit P2SH-P2WPKH Transactions **4**
- 3.1 Code . . . 4
- 3.2 Execution Walkthrough . . . 4
- 3.3 SegWit Script Analysis (P2SH-P2WPKH) . . . 4
  - 3.3.1 Structure of `scriptPubKey` . . . 4
  - 3.3.2 Structure of `redeemScript` . . . 5
  - 3.3.3 Structure of `scriptSig` . . . 5
  - 3.3.4 Structure of Witness Data . . . 5
  - 3.3.5 Explanation of SegWit Characteristics . . . 6
- 3.4 Numeric Comparison Table . . . 6

4 GitHub Repository Structure **7**

5 Conclusion **7**

6 References **7**

1 Environment Setup

This lab was conducted using a Bitcoin Core node running in `regtest` mode.

- Bitcoin Core Version: 30.2
- Operating System: Windows 11
- RPC IP: 127.0.0.1
- RPC Port: 18443

The following configuration was used in the `bitcoin.conf` file:

```
regtest=1
server=1
rpcuser=user
rpcpassword=password
rpcallowip=127.0.0.1
fallbackfee=0.0002

[regtest]
rpcport=18443
```

Python dependencies were installed using:

```
pip install -r requirements.txt
```

2 Part 1: Legacy P2PKH Transactions

2.1 Code

The Python script `part1_p2pkh.py` implements the creation and broadcasting of Legacy Pay-to-Public-Key-Hash (P2PKH) transactions. It handles wallet creation/loading, address generation, funding, and a two-step transaction chain (A → B, B → C).

2.2 Execution Walkthrough

To execute the P2PKH transaction flow, ensure Bitcoin Core is running in regtest mode and the RPC credentials in `part1_p2pkh.py` match your `bitcoin.conf`.

```
python part1_p2pkh.py
```

Sample Terminal Output

```
--- Setting up wallet: lab_wallet ---
Created wallet: lab_wallet

1. Generating Legacy Addresses...
Address A: mhBUXjwfbBQ47SZwjGvCPwXXk3S83XKsM4
Address B: mzhWXeGZceSs1JnDrvmeY3XWSRHwdPW2CF
Address C: muh14XmAfGwABLAVB6hcNRgBEP6zowrzLe

Funding TXID: b54d0bd104ffa922f38789438cab1f7545a2257b3ebdd69f2ac74c5cd02b7b89
```

2.3 Script Analysis (P2PKH)

2.3.1 Structure of `scriptPubKey` (Locking Script)

For a P2PKH transaction, the locking script has the following structure:

```
OP_DUP OP_HASH160 <PubKeyHash> OP_EQUALVERIFY OP_CHECKSIG
```

- `OP_DUP`: Duplicates the top stack item (which will be the public key).
- `OP_HASH160`: Hashes the duplicated public key using SHA256 followed by RIPEMD160, resulting in `PubKeyHash`.
- `<PubKeyHash>`: The 20-byte hash of the recipient's public key.
- `OP_EQUALVERIFY`: Compares the computed `PubKeyHash` with the provided `PubKeyHash`. If they are not equal, the script fails.
- `OP_CHECKSIG`: Verifies the transaction signature against the public key and transaction data.

2.3.2 Structure of `scriptSig` (Unlocking Script)

The unlocking script for a P2PKH transaction has the following structure:

```
<Signature> <PublicKey>
```

- `<Signature>`: The digital signature generated by the sender's private key.
- `<PublicKey>`: The sender's public key from which the `PubKeyHash` was derived.

2.3.3 Script Evaluation: Unlocking + Locking Concatenation

When a transaction is validated, the `scriptSig` is executed first, followed by the `scriptPubKey`. Conceptually, they are concatenated and executed as a single script.

The stack evaluation proceeds as follows:

1. Push `<Signature>` and `<PublicKey>` from `scriptSig`.
   Stack: `[PublicKey, Signature]`
2. `OP_DUP`: Duplicates the top item (`PublicKey`).
   Stack: `[PublicKey, PublicKey, Signature]`
3. `OP_HASH160`: Hashes the duplicated public key.
   Stack: `[PubKeyHash, PublicKey, Signature]`
4. Push `<PubKeyHash>` from `scriptPubKey`.
   Stack: `[ExpectedPubKeyHash, PubKeyHash, PublicKey, Signature]`
5. `OP_EQUALVERIFY`: Compares the two hashes and removes them if they match.
6. `OP_CHECKSIG`: Uses the public key and signature to verify the transaction data.

For the transaction to be valid, the final stack must contain a single `TRUE` value.

2.3.4 btcdeb Validation Steps

`btcdeb` is a powerful debugging tool for Bitcoin scripts. To validate script execution for a P2PKH transaction:

1. Load the transaction:

```
btcdeb -tx <raw_transaction_hex>
```

2. Set the input index:

```
input 0
```

3. Execute step-by-step:

```
step
```

   This allows inspection of the stack state after each opcode.

2.3.5 Signature Validation

Signature validation in P2PKH (and generally in Bitcoin) involves the `OP_CHECKSIG` opcode.

It performs the following checks:

(a) **Public Key Recovery**: Verifies that the signature corresponds to the provided public key.

(b) **Message Hashing**: Hashes the transaction data and compares it with the signature.

If both checks succeed, `OP_CHECKSIG` pushes `TRUE` onto the stack, confirming a valid signature and ownership of the funds.

3 Part 2: SegWit P2SH-P2WPKH Transactions

3.1 Code

The Python script `part2_segwit.py` demonstrates the creation and broadcasting of Segregated Witness (SegWit) transactions using the Pay-to-Script-Hash-Pay-to-Witness-Public-Key-Hash (P2SH-P2WPKH) address type.

Similar to Part 1, the script performs wallet setup, address generation, funding, and a two-step transaction chain (A' → B', B' → C').

3.2 Execution Walkthrough

To execute the SegWit transaction flow, ensure Bitcoin Core is running in `regtest` mode and the RPC credentials in `part2_segwit.py` match those defined in the `bitcoin.conf` file.

```
python part2_segwit.py
```

Sample Terminal Output

```
--- Setting up wallet: lab_wallet_segwit ---
Created wallet: lab_wallet_segwit

Generating SegWit Addresses...

Funding TXID: 9
   dac96c29fd428c331dd20e2ca59b24bc586764fa3612f30947dcc05336d5d73

Step 1: A' -> B'
TXID: 9d7ddf93b190dad03968982830887c45c4382740ec0b2ac8ea55a5d670ece626

Step 2: B' -> C'
TXID: e0d24cb6b2118bea3a0c422ffcfa1e231f1e3ed5151a9e49b832fff2a0339724
```

3.3 SegWit Script Analysis (P2SH-P2WPKH)

3.3.1 Structure of `scriptPubKey`

For a P2SH-P2WPKH transaction, the `scriptPubKey` is a P2SH script that locks funds to the hash of a redeem script. Its structure is:

```
OP_HASH160 <redeemScriptHash> OP_EQUAL
```

- `OP_HASH160`: Hashes the next data push.
- `<redeemScriptHash>`: The 20-byte hash of the redeem script.
- `OP_EQUAL`: Checks if the hash of the pushed data matches the `redeemScriptHash`. If they match, `TRUE` is pushed onto the stack.

3.3.2 Structure of `redeemScript`

The redeem script for P2SH-P2WPKH is a P2WPKH script with the following structure:

```
0 <PubKeyHash>
```

- `0`: This represents the witness version byte.
- `<PubKeyHash>`: The 20-byte hash of the public key.

This `redeemScript` is what gets hashed to form the `redeemScriptHash` used in the `scriptPubKey`.

3.3.3 Structure of `scriptSig`

For P2SH-P2WPKH, the `scriptSig` contains only the redeem script:

```
<redeemScript>
```

This redeem script is pushed onto the stack to satisfy the condition in the `scriptPubKey`:

```
OP_HASH160 <redeemScriptHash> OP_EQUAL
```

3.3.4 Structure of Witness Data

In SegWit transactions, the unlocking data (signature and public key) is moved to the witness field instead of the traditional `scriptSig`.

For P2SH-P2WPKH transactions, the witness stack contains:

```
<Signature> <PublicKey>
```

- `<Signature>`: The digital signature generated by the sender's private key.
- `<PublicKey>`: The sender's public key.

3.3.5 Explanation of SegWit Characteristics

- **Why `scriptSig` is smaller:**
  In P2SH-P2WPKH, the `scriptSig` only contains the redeem script:

  ```
  0 <PubKeyHash>
  ```

  This is significantly smaller than the full unlocking script of a P2PKH transaction, which includes both the signature and the public key. The signature and public key are instead moved to the witness field.

- **How witness data is separated:**
  Witness data is stored separately from the main transaction structure. It is placed after the traditional transaction fields but before the `scriptSig`. This separation is key to how SegWit improves transaction efficiency.

- **How validation occurs in SegWit:**
  Validation in SegWit occurs in two stages:

  (a) **Script Validation (P2SH part)**
  The `scriptSig` containing the `redeemScript` is executed against the `scriptPubKey`. This verifies that the provided `redeemScript` matches the `redeemScriptHash`.

  (b) **Witness Validation (P2WPKH part)**
  If the first stage succeeds, the redeem script (`0 <PubKeyHash>`) is used to determine how the witness data should be interpreted.
  The witness stack:

  ```
  <Signature> <PublicKey>
  ```

  is then executed against a virtual script derived from the redeem script:

  ```
  OP_DUP OP_HASH160 <PubKeyHash> OP_EQUALVERIFY OP_CHECKSIG
  ```

  This is essentially the same verification process used in P2PKH transactions, but it operates on witness data rather than the `scriptSig`.

3.4 Numeric Comparison Table

Metric	Legacy P2PKH (A → B)	SegWit P2SH-P2WPKH (A' → B')
Raw Size (bytes)	225	215
Virtual Size (vbytes)	225	134
Weight Units	900	533

Table 1: Transaction Size Comparison

4 GitHub Repository Structure

The repository is organized as follows:

Project Directory Structure

```
bitcoin-lab/
 part1_p2pkh.py
 part2_segwit.py
 requirements.txt
 README.md
 report.pdf
```

5 Conclusion

This lab demonstrated the creation and validation of Bitcoin transactions using both Legacy P2PKH and SegWit P2SH-P2WPKH formats within a Bitcoin Core regtest environment.

Legacy transactions store signature data directly inside the transaction structure, while SegWit separates signature data into the witness field. This separation improves transaction efficiency and reduces effective transaction size.

The experiment successfully highlighted the advantages of SegWit including improved fee efficiency, reduced transaction weight, and the resolution of transaction malleability issues.

6 References

(a) Bitcoin Wiki, "Script." Available at: https://en.bitcoin.it/wiki/Script

(b) Bitcoin Wiki, "Segregated Witness." Available at: https://en.bitcoin.it/wiki/Segregated_Witness

(c) A. M. Antonopoulos, *Mastering Bitcoin: Programming the Open Blockchain*. O'Reilly Media, 2017.